

PRACTICAL : 02

Aim:

Implementation and Time analysis of linear and binary search algorithm.

1. Linear Search.

Algorithm:

Linear Search

Input: Array 'A' with 'n' elements and a search element 'key'.

Output :- Position of 'key' if found ; otherwise display "Element not found".

1. Start
2. Read 'n', array 'A', and search element 'key'.
3. Set $i = 0$.
4. Compare $A[i]$ with key.
5. if $A[i] = \text{key}$, display the position/index of the element.
6. Otherwise, increment i by 1.
7. Repeat steps 4 to 6 until $i = n$.
8. if the element is not found, display "Element not found".
9. Stop.

Code:

```
#include <iostream>
using namespace std;

int Linearsearch(int arr[], int n, int search)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == search)
        {
            return i;
        }
    }

    return -1;
}

int main()
{
    int
    n;

    cout << "Enter number of elements: ";
    cin >> n;

    int *arr = new int[n];

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++)
    {
        cin >>
        arr[i];
    }

    int search;

    cout << "Enter element to search: ";
```

```
cin >> search;

int index = Linearsearch(arr, n, search);

if (index != -1)
{
    cout << "Element found at index " << index << endl;
}
else
{
    cout << "Element not found" << endl;
}

delete[] arr;

return 0;
}
```

Output:

```
Enter number of elements: 6
Enter elements:
2 88 9 4 78 90
Enter element to search: 4
Element found at index 3
```

Time Complexity:

- Linear Search :-

- Best Case :- element found at index 0.
only 1 comparison is made.

$$TC(n) = 1$$

$$TC(n) = O(1)$$

- Worst Case :- element ~~found~~ at index last, or not present.

All n comparisons are made.

$$TC(n) = 2n + 2 + n + 1 + 1$$

$$= 3n + 4$$

$$TC(n) = O(n)$$

- Average Case :- element equally likely at any position $1 \dots n$.

$$TC(n) = (1 + 2 + 3 + \dots + n) / 2$$

$$= [n(n+1)/2] / n$$

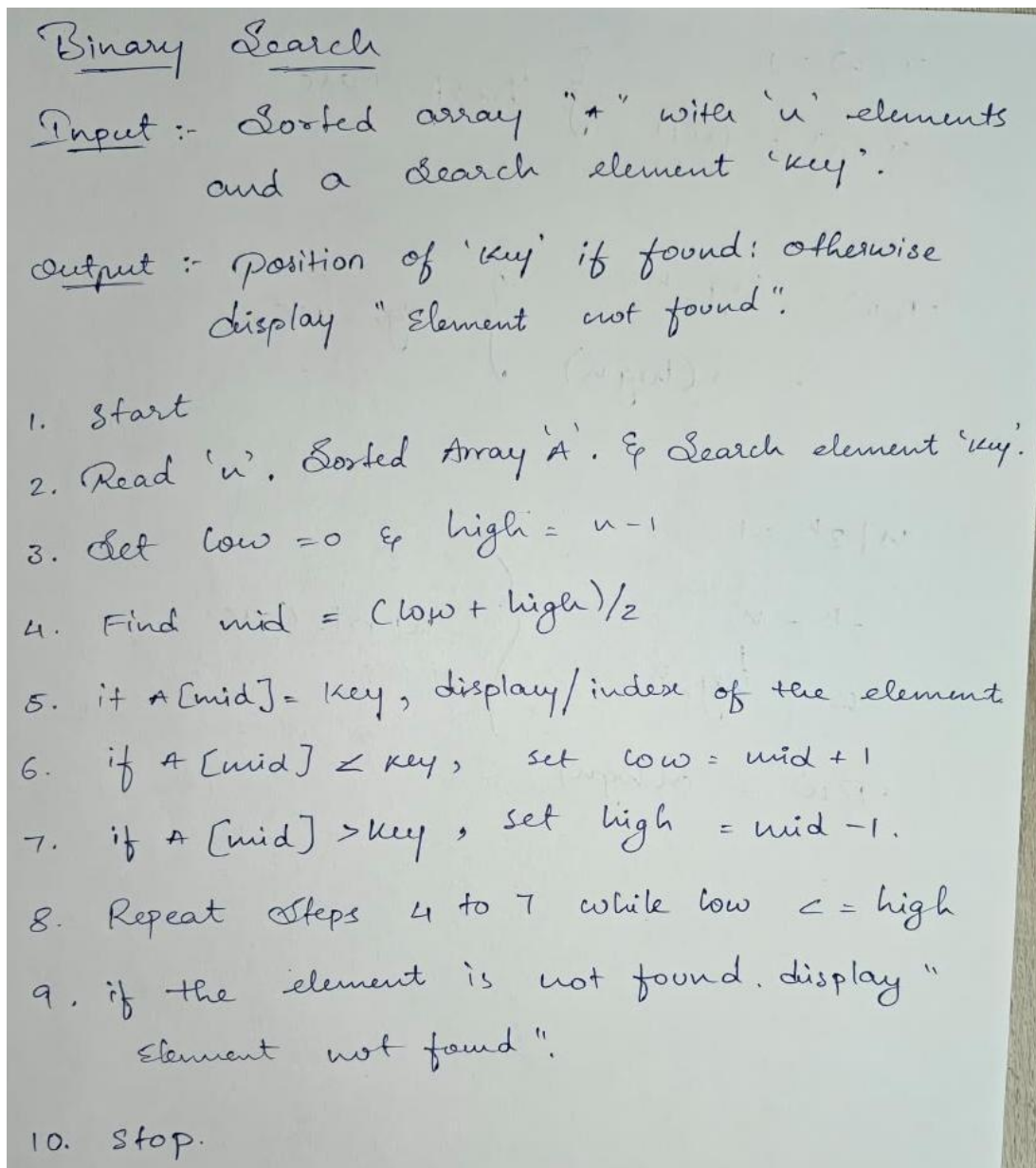
$$= (n+1)/2$$

$$TC(n) = (n+1)/2$$

$$= O(n)$$

2. Binary Search.

Algorithm:



Binary Search

Input :- Sorted array "A" with 'n' elements and a search element 'key'.

Output :- Position of 'key' if found; otherwise display "Element not found".

1. Start
2. Read 'n', Sorted Array 'A'. & Search element 'key'.
3. Set low = 0 & high = n - 1
4. Find mid = (low + high) / 2
5. if A[mid] = key, display/index of the element
6. if A[mid] < key, set low = mid + 1
7. if A[mid] > key, set high = mid - 1.
8. Repeat steps 4 to 7 while low <= high
9. if the element is not found, display "Element not found".
10. Stop.

Code:

```
#include <iostream>
using namespace std;

int Binarysearch(int arr[], int n, int search)
{   int low =
0;
    int high = n - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (arr[mid] == search)
        {
            return mid;
        }
        else if (arr[mid] < search)
        {
            low = mid + 1;
        }
    }
    else
    {
        high = mid - 1;
    }
}

return -1;
}

int main()
{   int
n;

    cout << "Enter number of elements: ";
    cin >> n;
```

```
int arr[n];

cout << "Enter elements in sorted order:\n";
for (int i = 0; i < n; i++)
{
    cin >> arr[i];
}

int search;

cout << "Enter element to search: ";
cin >> search;

int index = Binarysearch(arr, n, search);

if (index != -1)
{
    cout << "Element found at index " << index << endl;
}
else
{
    cout << "Element not found" << endl;
}

return 0;
}
```

Output:

```
Enter number of elements: 4
Enter elements in sorted order:
23 44 67 88
Enter element to search: 88
Element found at index 3
```




Time Complexity:

• Binary Search

- Best Case :- element found at first mid.
only 1 Comparison is made.

$$T(n) = 1$$

$$T(n) = O(1)$$

- Worst Case :- element's at boundary / not present.

Each step halves the array :

$$T(n) = T(n/2) + 1$$

$$T(n) = T(n/2) + 1$$

$$\text{Let } n = 2^k \Rightarrow k = \log_2(n)$$

$$T(n) = T(n/2) + 1$$

$$= T(n/4) + 1 + 1$$

$$= T(n/8) + 1 + 1 + 1$$

...

$$= T(n/2^k) + k$$

$$= T(1) + \log_2 n$$

$$\therefore T(n) = O(\log n)$$

- Average case :- on avg. above $(\log_2 n) - 1$ Comparisons are needed

$$T(n) = O(\log n)$$